

hermes-agent Pull Requests 5761 and 5771

Summary of changes and review status · Repository: NousResearch/hermes-agent · Prepared 2026-10-07

In one paragraph

These two PRs add skills that direct the Hermes agent to first run MathCode, which converts a natural-language theorem into a formalized statement in Lean 4, then loops over Mathlib tactics to generate an actual Lean proof, and finally to drive the Manim library to produce an animated explainer video grounded in that verified proof. The Lean work is done by MathCode, an external tool the skill operates; Manim is only involved in the video half.

So the end-to-end pipeline the author built is: the user asks for an explainer on a theorem, the agent runs MathCode to get a verified Lean proof, then the agent writes and renders a Manim video grounded in that proof. The two skills cover the two halves, and the ffmpeg fix repairs a bug hit while rendering the second half. "Manim library directed to formalize in Lean" is the one part that is not happening.

What is done, itemized

PR 5761 (new skills, 3 files, 699 lines added)

- 1 Adds a `mathcode` skill that teaches the Hermes agent to install, configure, and run MathCode, an external Lean 4 formalization CLI.
- 2 The skill tells the agent to run MathCode in two stages: formalize the theorem into a Lean statement with a `sorry` placeholder, then issue a second command to replace `sorry` with a Mathlib-tactic proof.
- 3 The skill specifies the required flags, the Anthropic backend configuration, precise prompt wording, foreground-only execution, and cronjob dispatch for long runs.
- 4 Adds a `manim-mathcode-explainer` skill that teaches the agent to write and render a 2 to 3 minute ELI5 Manim video about the proven theorem, with per-scene narration from the text-to-speech tool.
- 5 That skill defines strict ELI5 vocabulary, a proven 10-scene narrative arc, per-scene code templates, a narration-first timing workflow, layout rules, and a list of production pitfalls.
- 6 Adds a README for the explainer skill listing prerequisites and pointing to a setup script that is not included.

PR 5771 (same three files plus one fix, 760 added, 18 removed)

- 7 Contains both commits from PR 5761 unchanged.
- 8 Edits the existing `manim-video` skill's `references/rendering.md` to replace the `-c` copy stitch with a re-encode, because stream copy silently truncated concatenated Manim output.
- 9 Adds a silent-audio pre-step so scenes with and without audio can be concatenated.
- 10 Adds a two-way `ffprobe` duration check to detect broken timestamps.
- 11 Adds a freeze-frame tail-pad workflow for when narration outlasts the video.
- 12 Changes the voiceover mux from `-shortest` to an explicit `-t` duration so audio is not clipped.

13 Adds a note claiming text-to-speech always outputs .ogg, which the reviewer found incorrect.

Review outcome, both PRs open

14 The reviewer flagged a skill-name collision with the existing manim-video skill, the missing setup script, oversized descriptions and missing frontmatter and tests, the wrong TTS claim, and that the `-c copy` command still remains in the main manim-video SKILL.md.

15 The triage bot recommended closing PR 5761 as a duplicate of PR 5771 and splitting the new skills into their own compliant PR.

Overview

Both pull requests are open, authored by jeannemtl, and target the **main** branch. They overlap heavily: PR 5771 contains both commits of PR 5761 plus one additional commit that edits an existing reference file. Both branch from the same base commit (972482e2).

	PR 5761	PR 5771
Title	feat(skills): add mathcode and manim-mathcode-explainer skills	fix(skills): fix manim-video skill ffmpeg rendering bugs
Branch	jeannemtl:feature/mathcode_manim_integration	jeannemtl:fix/manim-rendering-bugs
State	Open	Open
Commits	2	3 (the 2 from PR 5761 + 1)
Files	3 added	3 added, 1 modified
Lines	+699 / -0	+760 / -18
Labels	type/feature, P3, tool/skills, sweeper:*	type/bug, P3, tool/skills, sweeper:*
Link	github.com/NousResearch/hermes-agent/pull/5761	github.com/NousResearch/hermes-agent/pull/5771

PR 5761: add mathcode and manim-mathcode-explainer skills

Two commits adding three new files. Nothing is deleted. The commits are 2cd95c69 (mathcode skill) and c15d40f5 (explainer skill).

1. skills/autonomous-ai-agents/mathcode/SKILL.md (70 lines, new)

A new skill teaching the agent to install and drive MathCode, a terminal-based Lean 4 formalization CLI. Frontmatter: name: `mathcode`, version 1.0.1. It covers:

- Installation by cloning `math-ai-org/mathcode` and running its setup script.
- Switching from the default OpenAI Codex backend to Anthropic via the `.env` file (`MATHCODE_USE_OPENAI=0`, `AUTOLEAN_USE_CODEX=0`, `ANTHROPIC_API_KEY`).
- Always running with `-p` plus `--dangerously-skip-permissions` to bypass the interactive workspace-trust prompt.
- Writing unambiguous prompts. Example: "Jordan's Lemma" exists in both complex analysis and quantum computing, and a vague prompt produced a `.lean` file that conflated both and failed with a sorry placeholder.

- Splitting formalization from proof generation. Outputs land in LeanFormalizations/ as .lean and .eval.json files.
- A warning that heavy tactic proofs can run 50 to 60+ minutes and consume large amounts of API credit.
- Run in the foreground with a long timeout, or via the cronjob tool for true background work. Never use the terminal tool's background flag, which makes MathCode hang silently.

Formalization versus proof generation: what the split means

The mathcode skill tells the agent to run MathCode in two separate stages rather than one command. They differ in what they produce, how long they take, and what they cost.

- 1 **Formalization.** MathCode reads the natural-language prompt and writes the theorem as a Lean 4 statement. The proof body is left as `sorry`, Lean's placeholder meaning "accept this unproven for now." The file type-checks, so the statement is well-formed and uses real Mathlib definitions, but nothing is verified yet. This stage is fast and cheap. It is also where ambiguity bites: the Jordan's Lemma example in the skill is a formalization failure, where a vague prompt produced a statement mixing complex analysis with quantum computing.
- 2 **Proof generation.** A second command points MathCode at the saved file and asks it to replace `sorry` with a real proof using Mathlib tactics. MathCode loops: write a candidate, compile, read the errors, revise. Each iteration is a model call plus a compiler run, which is why the skill warns it can take 50 to 60+ minutes and consume heavy API credit.

Output files in LeanFormalizations/

- **.lean** is the Lean source for the theorem. After stage 1 it holds the formal statement with `sorry` in place of the proof. After stage 2, if proving succeeded, it holds the full proof with no `sorry` left. MathCode's bundled axiom checker references a `problem_foo_proven`. Lean naming pattern for the finished version, and it scans files for leftover `sorry`, `admit`, or injected axiom declarations that would fake a proof.
- **.eval.json** is a machine-readable record written alongside the .lean file. The hermes skill only says it exists. The MathCode engine that writes it is a compiled binary and the schema is not documented in the public source or docs. Based on the surrounding tooling it is the evaluation record for that formalization (compile result, verification status, similar metadata). Treat the exact fields as unconfirmed.

Statement versus proof. A Lean theorem has two parts. The statement is everything before `:= by`: the variables, hypotheses, and conclusion. The proof is everything after. Formalization produces only the statement, with `sorry` in place of the proof:

```
theorem even_sq_even (n : ℕ) (h : Even n) : Even (n ^ 2) := by    -- the statement
  obtain {k, hk} := h                                           -- the proof
  refine (2 * k ^ 2, ?_)
  rw [hk]
  ring
```

The reason the two are treated as separate jobs is that they can fail independently, and Lean only catches one kind of failure:

- **A wrong statement with a correct proof** is the dangerous case. If MathCode formalizes the Jordan's Lemma prompt as some easier or unrelated claim, and then proves that claim, Lean will report success. The proof is valid, but it proves the wrong thing. Lean cannot tell you the statement does not match the English prompt. Only a human reading the statement can.

- **A correct statement with no proof** is the safe failure. The sorry stays, the axiom checker flags it, and everyone knows the theorem is unverified.

Why the split matters

- The .lean statement can be read after stage 1 and a wrong formalization caught in minutes, before spending an hour proving the wrong thing.
- The expensive stage can be dispatched separately. The skill says to run proving through the cronjob tool rather than in the foreground, where a new chat message interrupts it (exit code 130).
- Failures are easier to attribute. A file that will not compile is a formalization problem. A file that compiles but keeps its sorry is a proving problem.

Per MathCode's own documentation, a proof counts as complete only when its LeanVerify check returns verified. A .lean file with no sorry is necessary but is not the certificate by itself.

```
# Stage 1: formalize
cd ~/mathcode
./run -p "Formalize and prove Jordan's Lemma for pairs of reflections on a
finite-dimensional Hilbert space (as used in Grover's Algorithm)" --dangerously-skip-permissions

# Stage 2: prove (long-running; dispatch via cronjob tool)
./run -p "Write the full Lean 4 proof for the theorem in LeanFormalizations/problem_problem.lean.
Replace the 'sorry' with the actual proof using Mathlib tactics." --dangerously-skip-permissions
```

2. skills/creative/manim-mathcode-explainer/README.md (39 lines, new)

Installation guide listing the prerequisites: MathCode, Manim (pip), and ffmpeg. It explicitly says **do not install LaTeX**, because the pipeline uses Cairo Text () only and MathTex would crash. It also points to a verification script at scripts/setup.sh, which is not included in the PR.

3. skills/creative/manim-mathcode-explainer/SKILL.md (590 lines, new)

The actual pipeline for producing 2 to 3 minute "explain like I'm five" theorem videos. Its frontmatter declares name: manim-video (see review issues). Main sections:

- **Creative standard.** Narrative arc first, geometry before algebra, opacity layering, one cohesive visual language.
- **ELI5 vocabulary rules.** Only physical-object words (mirrors, flips, spins, doors). Terms such as self-adjoint, unitary, eigenvalue, or any mention of Lean are rejected. Includes the calibration history from the Jordan's Lemma video.
- **Proven 10-scene arc** with target durations: hook, core primitive, two primitives, reveal, rule, lemma statement, why always true, real-world stakes, mechanism, recap. Total about 2m10s.
- **Scene code templates** for each scene, with beat maps that tie animation timing to the narration.
- **Per-scene audio workflow** (required): generate narration per scene first, measure it, then write wait values to match.
- **Layout rules:** three-zone title-card-then-animation layout, no floating labels near moving objects, simple animations.
- **Critical pitfalls** learned in production, such as MoveAlongPath combined with .animate producing invisible motion, self.wait(0) raising, the Manim timestamp bug truncating concatenated video, and mixed-audio concat failures.

- **Full pipeline, render commands, performance targets, and color palette.**

The PR description says the workflow was validated end to end on macOS 15 (Apple Silicon M4, Python 3.9, Manim CE 0.18) with three videos: Jordan's Lemma, No-Cloning Theorem, and Pythagorean Theorem.

PR 5771: fix manim-video ffmpeg rendering bugs

Contains the same two commits as PR 5761, plus a third commit (327d563c) that modifies one existing file: skills/creative/manim-video/references/rendering.md (61 lines added, 18 removed). The motivation: three silent failures found while producing the Jordan's Lemma and No-Cloning videos with the pipeline from PR 5761.

What the third commit changes

- **Replaces the lossless stitch.** The old step used `ffmpeg -f concat -c copy`. Manim partial files carry non-monotonic DTS timestamps, so stream copy silently truncated output (74 s produced when 104 s was expected, no ffmpeg error). The new step re-encodes.
- **Adds a silent-audio pre-step.** Each scene gets a silent AAC track muxed in first, so scenes with and without audio can be concatenated together.
- **Adds a duration check.** Two `ffprobe` calls, one for container duration and one for the video stream, because they disagree when timestamps are broken. If they differ, re-encode.
- **Adds a tail-pad section.** When the voiceover is longer than the video, grab the last frame, loop it into a filler clip, and concatenate. The pad fps must match the scene fps (15 for -ql, 30 for -qm, 60 for -qh) or `concat` silently drops it.
- **Rewrites the voiceover mux.** Uses `-t "$AUDIO_DUR"` instead of `-shortest`, which clipped audio whenever the video ended first. The old per-scene audio concat example is removed.
- **Adds a note** claiming the `text_to_speech` tool always outputs .ogg regardless of the requested extension.
- Also removes a handful of blank lines in otherwise untouched sections below (manim.cfg, Sections, manim-voiceover).

Before and after, stitch step

```
# Before
ffmpeg -y -f concat -safe 0 -i concat.txt -c copy final.mp4

# After
for scene in Scene1 Scene2 Scene3; do
  ffmpeg -y -i "media/videos/script/480p15/${scene}.mp4" \
    -f lavfi -i anullsrc=channel_layout=stereo:sample_rate=44100 \
    -c:v copy -c:a aac -shortest "media/videos/script/480p15/${scene}_s.mp4"
done
ffmpeg -y -f concat -safe 0 -i concat.txt \
  -vf "fps=15" -c:v libx264 -preset fast -crf 22 -c:a aac stitched.mp4
```

Before and after, voiceover mux

```
# Before
ffmpeg -y -i final.mp4 -i narration.mp3 -c:v copy -c:a aac -b:a 192k -shortest final_narrated.mp4

# After
AUDIO_DUR=$(ffprobe -i voiceover.ogg -show_entries format=duration -v quiet -of csv="p=0")
ffmpeg -y -i full_video.mp4 -i voiceover.ogg \
  -c:v copy -c:a aac -map 0:v:0 -map 1:a:0 -t "$AUDIO_DUR" final_with_audio.mp4
```

Review status

Collaborator teknum1 reviewed both PRs with the same core objections. The triage bot GottZ noted that PR 5771 duplicates PR 5761 and recommended closing 5771 as a duplicate. The blocking items:

- 1 **Skill name collision.** The new explainer SKILL.md declares name: `manim-video`, identical to the existing bundled skill at `skills/creative/manim-video/SKILL.md`. The skill loader silently skips duplicates, so the new skill is never discoverable. It needs a unique name matching its directory.
- 2 **Missing file.** The README references `scripts/setup.sh`, which neither PR includes.
- 3 **Metadata non-compliance.** Descriptions run 271 and 452 characters against a 60-character limit. Frontmatter lacks the required author and platform fields. No tests exist at `tests/skills/test_<skill>_skill.py`. Reference: AGENTS.md lines 888 to 940.
- 4 **Incorrect TTS claim.** Verified against current `tools/tts_tool.py`: output is Opus .ogg only on voice-bubble platforms or when .ogg is explicitly requested, and MP3 otherwise. The "always .ogg" note is wrong.
- 5 **Incomplete fix.** Verified on current main: the old `-c copy concat` command still sits in the primary workflow at `skills/creative/manim-video/SKILL.md` line 185 (Step 4: Stitch). PR 5771 only fixes the `references/rendering.md` copy.

Suggested path forward

teknum1 recommends keeping PR 5771 as a focused fix that applies the corrected concat guidance to both Manim workflows (SKILL.md and `references/rendering.md`), with the TTS note corrected. The two new skills should move to a separate PR with unique directory-matching names, compliant frontmatter (short description, author, platform), the missing setup script, and skill tests. PR 5761 would then be closed as a duplicate.

Appendix: the mathcode skill as itemized executed steps

The mathcode skill (`skills/autonomous-ai-agents/mathcode/SKILL.md`) reduced to what it actually instructs the agent to execute, in order. Each line is a declarative statement of a step or rule from the file. Items marked "Rule" constrain behavior but execute nothing on their own.

Install

- 1 The agent clones `https://github.com/math-ai-org/mathcode.git` into `~/mathcode`.
- 2 The agent runs `bash setup.sh` inside `~/mathcode`.

Configure (Anthropic backend)

- 3 The agent writes `MATHCODE_USE_OPENAI=0` to `~/mathcode/.env`, which disables Codex OAuth.
- 4 The agent writes `AUTOLEAN_USE_CODEX=0` to `~/mathcode/.env`, which disables the Codex backend for the math tools.
- 5 The agent writes `ANTHROPIC_API_KEY=sk-ant-...` to `~/mathcode/.env`.
- 6 Rule: shell-exported environment variables override the `.env` file.

Invoke

- 7 The agent runs MathCode as `./run -p "<prompt>" --dangerously-skip-permissions` from `~/mathcode`.
- 8 Rule: `-p` is always present, which skips the interactive UI.
- 9 Rule: `--dangerously-skip-permissions` is always present, which bypasses the workspace-trust dialog during tool execution.
- 10 Rule: the prompt is never piped on stdin (`echo "..." | ./run -p`); it is passed as the `-p` argument.

Prompt construction

- 11 Rule: the prompt names the theorem with its domain and setting, for example "Jordan's Lemma for pairs of reflections on a finite-dimensional Hilbert space (as used in Grover's Algorithm)" rather than "Jordan's Lemma (Grover's Algorithm)."
- 12 Rule: the reason is that MathCode conflates same-named theorems from different fields and emits a `.lean` file that mixes domains and fails with `sorry`.

Stage 1: formalize

- 13 The agent issues a `./run -p` call asking MathCode to formalize the theorem.
- 14 MathCode writes the Lean 4 statement, with `sorry` as the proof body, to `~/mathcode/LeanFormalizations/<name>.lean`.
- 15 MathCode writes a companion `~/mathcode/LeanFormalizations/<name>.eval.json`.
- 16 **Not executed:** the skill contains no step that reads, validates, or asks the user to confirm the generated statement.

Stage 2: prove

- 17 The agent issues a second `./run -p` call whose prompt points at the saved file: "Write the full Lean 4 proof for the theorem in `LeanFormalizations/<name>.lean`. Replace the 'sorry' with the actual proof using Mathlib tactics."
- 18 Rule: the agent does not run MathCode's underlying scripts directly to produce the proof; it always goes through the `run` binary.
- 19 Rule: the agent expects this call to take 50 to 60+ minutes and to consume large amounts of Anthropic API credit.
- 20 If the MathCode process disappears from `ps aux` without updating the file, the agent checks the Anthropic API balance, since the likely cause is a "credit balance is too low" error.

Process management (foreground)

- 21 The agent runs MathCode as a blocking foreground terminal call with a high timeout, 600 or 900 seconds given as examples.
- 22 Rule: the agent never uses the terminal tool's `background=true`, because MathCode then hangs or sleeps indefinitely.
- 23 Before starting a long foreground call, the agent tells the user: "Please do not send any messages while this is running, as it will interrupt the proof generation."
- 24 Rule: a user message during a foreground call kills it with exit code 130.

Process management (background, on request)

- 25 If the user wants to keep chatting while MathCode runs, the agent dispatches the run through the `cronjob` tool, which launches a separate agent in an isolated session.
- 26 Rule: the agent does not use `delegate_task` for this, because it still blocks the current chat turn until the subagent finishes.
- 27 Rule: `cronjob` logs fill with MathCode's loading-spinner frames; the agent treats this as normal and ignores it.
- 28 To track progress of a `cronjob` run, the agent runs `ps aux | grep autolean` to see whether the worker is alive.
- 29 To read actual progress, the agent uses `search_files` and `read_file` on the logs under `/var/folders/.../AUTOLEAN/logs/` or wherever the temp working directory was created.

Observations

- Items 1 through 5 are one-time setup. Items 7 through 29 are the per-theorem loop.
- The only places the skill tells the agent to communicate with the user are item 23, the "do not send messages" warning, and item 25, which is user-initiated. Nothing asks the user to review the statement or the finished proof.